

• Section- A (2×5=10)

1. Explain error detection and recovery in lexical phase.

Ans • Detection - Errors like illegal characters, invalid identifiers or incorrect tokens.

• Recovery - Skipping invalid characters, inserting missing tokens, or using predefined corrections.

2. Explain error detection and recovery in syntactic phase.

Ans Detection - Errors in syntax like missing parentheses, misplaced keywords.

• Recovery - Panic-mode (skipping tokens), Phrase-level (modifying input) error productions.

3. Explain error detection and recovery in semantic phase.

Ans Detection - Type mismatches, undefined variable, incorrect function calls.

• Recovery - Type conversion, default values, reporting errors with hints.

4. What is activation record?

Ans An activation record is a data structure used in the runtime stack to manage function calls and store relevant information during program execution.

5. Discuss various fields of activation record.

Ans Fields of Activation Records -

• Return Address - Store where control should return.

• Local Variables - Stores function's local data.

• Parameters - Stores arguments passed to function.

• Temp variables: - Holds intermediate values.

• Control Links - Connects activation records in stack.

• Section-B (7x3=21)

1. What are the symbol table requirements? What are the demerits in the uniform structure of symbol table?

Ans A symbol table is a data structure used by a compiler to store information about variables, functions, objects, and other program identifiers. The key requirements are:

- Efficient insertion & Lookup - Fast retrieval of symbol information.
- Scope Management - Tracks identifiers in different scopes (global, local).
- Handling Overloading & Inheritance - Support polymorphism and inheritance in OOP.
- Memory Optimization - Efficiently stores symbols without excessive space usage.
- Error Handling - Helps detect undeclared variables.

• Demerits of Uniform structure in Symbol Table (20)

A uniform structure means a single table format for all symbols, which has drawbacks:

- (i) Inefficient Access - Different symbols (variables, functions) require different handling, leading to slower access.
- (ii) Increased Memory Usage - Uniform storage may waste space when storing small symbols.
- (iii) Difficulty in Type Handling - Different types (array, structure, function) may need extra processing.
- (iv) Limited Optimization - Harder to implement separate optimizers for different data categories.

2. Differentiate between lexical scope and dynamic scope.

Ans Here is the difference between Lexical scope and Dynamic scope.

Features	Lexical Scope (Static scope)	Dynamic scope
• Definition	Scope of a variable is determined at compile time, based on the structure of the code.	Scope of a variable is determined at runtime based on the function call sequence.
• Resolution	Uses the block structure of the program to find variable decl.	Uses the call stack to find variable declarations.
• Efficiency	More efficient, as variable locations are known at compile time.	Less efficient, as variable resolution happens at runtime.
• Usage	Used in most modern languages (C, Java, Python)	Used in older lang. like Lisp and early Basic.
• Example	in C: $\{ \text{int } x=10; \{ \text{int } y=x+5; \} \}$ → x is resolved statically.	in Lisp: A function access a variable from the caller's scope at runtime.

3. Discuss all the data structures used for implementing the symbol table.

Ans. A symbol table is implemented using various data structures to efficiently store and retrieve symbol information. The commonly used data structures are:

(i) Linear List ~~Table~~ - Symbols are stored in a simple list.

• Pros - Easy to implement.

• Cons - Slow lookup time ($O(n)$).

(ii) Hash Table - Uses hash functions to map symbols to an index in an array.

• Pros - Fast lookup ($O(1)$) in average case.

• Cons - Collisions requires handling.

(iii) Binary Search Tree (BST) - stores symbols in a sorted tree for efficient searching.

• Pros - Faster lookup than a list ($O(\log n)$).

• Cons - Performance degrades to $O(n)$ in unbalanced trees.

(iv) Trie (Prefix Tree) - Stores symbols character-by-character in a tree structure.

Pros - Efficient for prefix-based lookup.

Cons - High memory usage.

(v) Graph-Based Structure - Used in DOP lang. where interrelationships and scope create relationships betⁿ symbols.

Pros - Handles complex relationships efficiently.

Cons - Harder to implement.

• Section-C (11x3=33)

Q. What is symbol table and what kind of information it stores? Discuss its capabilities and use of symbol table.

Ans. A symbol table is a data structure used by the compiler to store and manage information about variables, functions, objects and other identifiers in a program. It helps in efficient semantic analysis, type checking and memory allocation during compilation.

Information stored in a Symbol Table:-

A symbol table stores various details about each identifier including:

- (i) Name - The variable or function name.
- (ii) Type - Data Type (int, float, char, etc).
- (iii) Scope - Global, local, or block scope.
- (iv) Memory location - Address or offset where the variable is stored.
- (v) Value - Constant values assigned to identifiers.
- (vi) Function Parameters - Number and Type of arguments for function.
- (vii) Attributes - Additional properties like static, extern or volatile.

Capabilities of Symbol Table -

- (i) Efficient Lookup and Retrieval - Quickly finds symbols in the source code.
- (ii) Scope Management - Maintains variable scope and prevents conflicts.
- (iii) Type Checking - Ensures type consistency in expressions and function calls.
- (iv) Error Detection - Identifies undeclared variables and multiple declarations.
- (v) Optimization Support - Helps in eliminating redundant computations.

Uses of Symbol Table :-

- (i) Compiler Phases - Used in lexical analysis, syntax analysis and semantic analysis.
- (ii) Code Optimization - Stores reusable expressions to optimize computations.
- (iii) Memory Management - Allocates and deallocates memory for variables dynamically.
- (iv) Debugging & Error Handling - Helps in tracking errors related to scope and type mismatches.

2. Write down the operations performed on a symbol table.
 Ans A symbol table supports various operations to manage identities efficiently during compilation. The key operations are :-

- (i) Insertion - Adds a new symbol (variable, function, class) to the table.
 - Stores relevant attributes like name, type, scope and memory location.
- (ii) Lookup / search -
 - Checks if a symbol exists in the table.
 - Used during semantic analysis to verify declarations.

(iii) Modifications / update -

- Updates attributes of an existing symbol.
- Useful in languages supporting dynamic typing.

(iv) Deletion -

- Removes symbols when they go out of scope.
- Helps in memory management.

(v) Scope Management -

- Handles nested and global scopes using stack-based structures.
- Ensures correct identifier resolution.

(vi) Traversal -

- Iterates through the symbol table to perform tasks like garbage collection, code optimization, or error checking.

3. Discuss how the scope information is represented in a symbol table.

Ans - Scope defines the visibility and lifetime of variables and functions in a program. A symbol table must efficiently manage scope information to prevent name conflicts and ensure proper variable resolution.

Methods for Representing Scope in a Symbol Table.

(i) Flat Symbol Table -

- A single table stores all symbols.
- Used for small programs but inefficient for nested scopes.

(ii) Hierarchical (Tree-based) Symbol Table -

- Uses a tree where each scope is a node.

- Child nodes inherit symbols from parent nodes.
- Efficient for block-structured languages (e.g., C, Java, Python).

(iii) Linked List of Symbol Table :-

- Each scope has its own table, linked to its parent.
- Lookup starts from the current scope and moves upward.
- Used in dynamic languages (e.g., JS, Lisp).

(iv) Stack-based Symbol Table -

- Uses a stack where a new scope pushes a table onto the stack.
- When exiting a scope, it pops the table.
- Efficient for block-scoped languages like C and Java.

(v) Hash Table with Scope Numbering -

- Uses a hash table where symbols are stored with scope number.
- Helps in fast lookup without traversing a hierarchy.

